jQuantStats: Portfolio Analytics for Quants

A Companion Paper

Jebel Quant

March 2026

Abstract

jQuantStats is an open-source Python library for portfolio performance analytics designed for systematic traders and quantitative portfolio managers. It provides a comprehensive suite of over eighty risk and return metrics, interactive Plotly visualisations, and an HTML reporting engine, all built on a modern, Polars-native stack with zero pandas runtime dependency.

The library exposes two entry points: `Portfolio`, which ingests raw price series and position sizes to compute transaction-cost-aware performance metrics, and `Data`, which operates on pre-computed return streams. Both surfaces share a unified `Stats` facade that covers descriptive statistics, drawdown analysis, risk-adjusted ratios, benchmark attribution, and rolling analytics.

This paper documents the design rationale, mathematical foundations, and practical usage of the library. It serves as a reference for practitioners who wish to understand the precise definitions underlying each metric, the Brinson-style tilt/timing attribution model, the two-factor transaction cost framework, and the conventions used to annualise statistics across different return frequencies.

Contents

1 Introduction

Quantitative practitioners routinely need to evaluate a strategy's historical performance across many dimensions: raw return, risk-adjusted return, drawdown characteristics, benchmark sensitivity, concentration, and trading costs. Existing open-source tools such as `QuantStats` (Aroussi, 2019) were built on pandas and matplotlib and reflect the conventions of an earlier era of Python data science.

jQuantStats was written from scratch to address three structural limitations of that generation of tools.

1. **DataFrame backend.** Pandas carries significant overhead for columnar workloads that arise in portfolio analytics. jQuantStats uses Polars (Polars Development Team, 2024) as its primary engine, reducing memory usage and improving throughput on large price histories. The public API is mediated through Narwhals (Narwhals Development Team, 2024), a lightweight abstraction layer that accepts both Polars and pandas frames at system boundaries.

2. **Visualisation.** Static matplotlib figures are unsuitable for interactive analysis. Every chart in jQuantStats is rendered with Plotly, producing fully interactive HTML widgets that can be embedded in notebooks, dashboards, or self-contained HTML reports.

3. **Type safety.** The library targets Python 3.11+, is fully annotated, ships a `py.typed` marker, and achieves 100% docstring coverage. This makes IDE integration and downstream static analysis straightforward.

The remainder of this paper is structured as follows. Section 2 describes the two-class architecture and data model. Section 3 derives the mathematical definitions of every metric. Section 4 presents the Brinson tilt/timing attribution framework. Section 5 documents the transaction cost models. Section 6 surveys the visualisation and reporting surfaces. Section 8 gives an end-to-end worked example. Section 7 discusses frequency detection and annualisation conventions. Appendix A provides a compact API reference.

2 Architecture and Data Model

2.1 Entry Points

The public API consists of two classes.

Portfolio. `Portfolio` is the primary entry point for systematic strategies where the analyst controls both the signal and the position sizing. It is constructed via

```
from jquantstats import Portfolio

pf = Portfolio.from_cash_position(
    prices=prices_df,        # Polars DataFrame, one column per instrument
    cash_position=pos_df,    # signed cash notional per instrument per day
    aum=1_000_000,           # total assets under management
    cost_per_unit=0.0,       # optional: GBP per share traded
    cost_bps=0.0,            # optional: basis points of AUM turnover
)
```

Listing 1: Constructing a Portfolio from prices and positions.

`Portfolio` derives all downstream quantities—daily profit and loss, net asset value, returns, drawdown series, tilt/timing decomposition, and turnover—directly from the raw inputs without requiring the caller to pre-process data.

Data. `Data` is a lighter entry point for analysts working with pre-computed return streams, for example when evaluating a signal provided by a third party.

```python
from jquantstats import Data

d = Data.from_returns(
    returns=returns_series,      # Polars Series of period returns
    rf=0.0,                      # risk-free rate (same frequency as returns)
    benchmark=bench_series,      # optional benchmark returns
)
```

Listing 2: Constructing a Data object from returns.

2.2 Internal Data Flow

Figure 1 illustrates how raw inputs flow through the library.

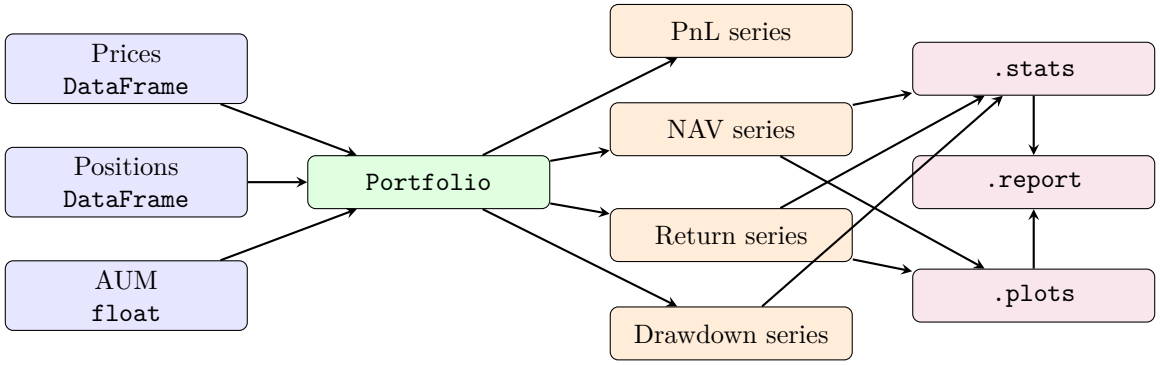

Figure 1: Data flow from raw inputs through `Portfolio` to the analytics facades.

2.3 Facade Pattern

Both `Portfolio` and `Data` expose three facade properties: `.stats`, `.plots`, and `.report` (or `.reports` on `Data`). These facades are thin delegation objects constructed lazily on first access. This design keeps the primary classes lightweight while providing rich secondary surfaces.

3 Statistical Metrics

Let $\{r_t\}_{t=1}^{T}$ denote the time series of T period returns and let f denote the number of periods per year (e.g. $f = 252$ for daily returns, $f = 52$ for weekly, $f = 12$ for monthly). Unless stated otherwise, all series are excess returns net of a risk-free rate r_f.

3.1 Descriptive Statistics

Average return.

$$\bar{r} = \frac{1}{T} \sum_{t=1}^{T} r_t. \tag{1}$$

Volatility. Annualised realised volatility is

$$\sigma = \sqrt{f} \cdot \sqrt{\frac{1}{T-1} \sum_{t=1}^{T} (r_t - \bar{r})^2}. \tag{2}$$

Skewness and excess kurtosis.

$$\text{skew} = \frac{1}{T}\sum_{t=1}^{T}\left(\frac{r_t - \bar{r}}{\hat{\sigma}}\right)^3, \qquad \text{kurt} = \frac{1}{T}\sum_{t=1}^{T}\left(\frac{r_t - \bar{r}}{\hat{\sigma}}\right)^4 - 3, \tag{3}$$

where $\hat{\sigma}$ is the sample standard deviation (without the $\sqrt{f}$ annualisation factor).

Win rate.

$$\text{WinRate} = \frac{|\{t : r_t > 0\}|}{T}. \tag{4}$$

Payoff and profit ratios. Let $W = \{t : r_t > 0\}$ and $L = \{t : r_t < 0\}$. Then

$$\text{PayoffRatio} = \frac{\bar{r}_W}{|\bar{r}_L|}, \qquad \text{ProfitFactor} = \frac{\sum_{t\in W} r_t}{|\sum_{t\in L} r_t|}, \tag{5}$$

where $\bar{r}_W$ and $\bar{r}_L$ are the conditional averages over winning and losing periods respectively.

Gain-to-pain ratio.

$$\text{GtP} = \frac{\sum_{t=1}^{T} r_t}{\sum_{t\in L} |r_t|}. \tag{6}$$

Exposure. The fraction of periods in which the strategy holds a non-zero position:

$$\text{Exposure} = \frac{|\{t : r_t \neq 0\}|}{T}. \tag{7}$$

3.2 Risk Metrics

Value at Risk. The parametric Cornish–Fisher VaR at confidence level α is

$$\text{VaR}_\alpha = -\left(\bar{r} + z_\alpha^{\text{CF}} \cdot \hat{\sigma}\right), \tag{8}$$

where the Cornish–Fisher expansion (Cornish and Fisher, 1937) adjusts the standard normal quantile z_α for skewness s and excess kurtosis k:

$$z_\alpha^{\text{CF}} = z_\alpha + \frac{z_\alpha^2 - 1}{6}\, s + \frac{z_\alpha^3 - 3z_\alpha}{24}\, k - \frac{2z_\alpha^3 - 5z_\alpha}{36}\, s^2. \tag{9}$$

Conditional Value at Risk (Expected Shortfall).

$$\text{CVaR}_\alpha = -\mathbb{E}[r_t \mid r_t \leq -\text{VaR}_\alpha]. \tag{10}$$

Kelly criterion. The continuous-time Kelly fraction under a Gaussian return assumption is

$$k^* = \frac{\bar{r}}{\hat{\sigma}^2}. \tag{11}$$

3.3 Drawdown Analysis

Let $p_t = \prod_{s=1}^{t}(1 + r_s)$ denote the NAV indexed to 1 at inception. The *high-water mark* is

$$h_t = \max_{s\leq t} p_s, \tag{12}$$

and the drawdown at time t is

$$d_t = \frac{h_t - p_t}{h_t} \;\in\; [0, 1). \tag{13}$$

Maximum drawdown.

$$\mathrm{MDD} = \max_{t \le T} d_t. \tag{14}$$

Maximum drawdown duration. The longest continuous interval $[t_1, t_2]$ such that $p_t < h_{t_1 - 1}$ for all $t \in [t_1, t_2]$.

Average drawdown. The mean drawdown depth over all underwater periods.

3.4 Risk-Adjusted Ratios

Sharpe ratio (Sharpe, 1966).

$$\mathrm{SR} = \frac{\bar{r}}{\hat{\sigma}/\sqrt{f}} = \frac{\bar{r}\sqrt{f}}{\hat{\sigma}}. \tag{15}$$

Asymptotic Sharpe variance. Under mild regularity conditions (Bailey and López de Prado, 2012) the sample Sharpe ratio $\widehat{\mathrm{SR}}$ has variance

$$\mathrm{Var}(\widehat{\mathrm{SR}}) \approx \frac{1}{T}\left(1 + \frac{\mathrm{skew} \cdot \widehat{\mathrm{SR}}}{2} + \frac{(\mathrm{kurt} - 3) \cdot \widehat{\mathrm{SR}}^2}{4}\right). \tag{16}$$

Probabilistic Sharpe ratio. The probability that the true Sharpe ratio exceeds a benchmark SR^*:

$$\mathrm{PSR}(\mathrm{SR}^*) = \Phi\left(\frac{(\widehat{\mathrm{SR}} - \mathrm{SR}^*)\sqrt{T-1}}{\sqrt{1 - \mathrm{skew} \cdot \widehat{\mathrm{SR}} + \frac{\mathrm{kurt}-1}{4} \cdot \widehat{\mathrm{SR}}^2}}\right), \tag{17}$$

where Φ is the standard normal CDF.

Sortino ratio (Sortino and Price, 1994).

$$\mathrm{Sortino} = \frac{\bar{r}\sqrt{f}}{\sigma_d}, \qquad \sigma_d = \sqrt{\frac{1}{T}\sum_{t=1}^{T} \min(r_t, 0)^2}. \tag{18}$$

Calmar ratio (Young, 1991).

$$\mathrm{Calmar} = \frac{\bar{r}\, f}{\mathrm{MDD}}. \tag{19}$$

Recovery factor.

$$\mathrm{RF} = \frac{p_T - 1}{\mathrm{MDD}}. \tag{20}$$

Risk–return ratio.

$$\mathrm{RRR} = \frac{\bar{r}}{\mathrm{CVaR}_\alpha}. \tag{21}$$

3.5 Concentration Metrics

The Herfindahl–Hirschman Index (Herfindahl, 1950) applied to the cross-sectional distribution of signed returns:

$$\mathrm{HHI}^+ = \sum_{t \in W}\left(\frac{r_t}{\sum_{s \in W} r_s}\right)^2, \qquad \mathrm{HHI}^- = \sum_{t \in L}\left(\frac{r_t}{\sum_{s \in L} r_s}\right)^2. \tag{22}$$

Values close to $1/|W|$ (or $1/|L|$) indicate uniform dispersion; values close to 1 indicate concentration in a single period.

3.6 Benchmark and Factor Analytics

Let $\{b_t\}$ be a benchmark return series of the same length. Define active returns $a_t = r_t - b_t$.

Information ratio.

$$\mathrm{IR} = \frac{\bar{a}\sqrt{f}}{\hat{\sigma}_a}, \qquad \hat{\sigma}_a = \sqrt{\frac{1}{T-1}\sum_t (a_t - \bar{a})^2}. \tag{23}$$

Alpha and Beta (Jensen, 1968). Estimated by OLS of $r_t = \alpha + \beta\, b_t + \varepsilon_t$:

$$\hat{\beta} = \frac{\mathrm{Cov}(r_t,\, b_t)}{\mathrm{Var}(b_t)}, \tag{24}$$

$$\hat{\alpha} = \bar{r} - \hat{\beta}\,\bar{b}. \tag{25}$$

R-squared.

$$R^2 = 1 - \frac{\sum_t (r_t - \hat{\alpha} - \hat{\beta}\, b_t)^2}{\sum_t (r_t - \bar{r})^2}. \tag{26}$$

Capture ratios.

$$\mathrm{UpCapture} = \frac{\bar{r}_{b>0}}{\bar{b}_{b>0}}, \qquad \mathrm{DownCapture} = \frac{\bar{r}_{b<0}}{\bar{b}_{b<0}}, \tag{27}$$

where subscripts denote conditioning on benchmark return sign.

3.7 Compound Return Metrics

Compound return.

$$\mathrm{Comp} = \prod_{t=1}^{T}(1 + r_t) - 1. \tag{28}$$

Geometric holding period return (GHPR). The per-period geometric mean return, equivalent to the T-th root of the terminal wealth ratio:

$$\mathrm{GHPR} = \left(\prod_{t=1}^{T}(1 + r_t)\right)^{1/T} - 1. \tag{29}$$

Compound annual growth rate.

$$\mathrm{CAGR} = \left(\prod_{t=1}^{T}(1 + r_t)\right)^{f/T} - 1. \tag{30}$$

3.8 Omega Ratio

The Omega ratio (Keating and Shadwick, 2002) measures the probability-weighted ratio of gains to losses relative to a threshold return τ:

$$\Omega(\tau) = \frac{\int_\tau^\infty [1 - F(r)]\,\mathrm{d}r}{\int_{-\infty}^\tau F(r)\,\mathrm{d}r}, \tag{31}$$

where F is the empirical cumulative distribution of returns. In the discrete-sample implementation with $\tau = r_f$:

$$\Omega = \frac{\sum_{t:r_t > \tau}(r_t - \tau)}{\sum_{t:r_t \le \tau}(\tau - r_t)}. \tag{32}$$

3.9 Autocorrelation Metrics

Autocorrelation at lag ℓ.

$$\rho(\ell) = \frac{\sum_{t=\ell+1}^{T}(r_t - \bar{r})(r_{t-\ell} - \bar{r})}{\sum_{t=1}^{T}(r_t - \bar{r})^2}. \tag{33}$$

Autocorrelation penalty (Lo, 2002). Under serially correlated returns, the Sharpe ratio estimator has inflated variance. The autocorrelation penalty scales the Sharpe ratio to correct for this bias:

$$\text{penalty} = \sqrt{1 + 2 \sum_{\ell=1}^{\min(T-1,\, \lfloor T/5 \rfloor)} \rho(\ell)\, \frac{T-\ell}{T}}. \tag{34}$$

`smart_sharpe()` and `smart_sortino()` divide the standard Sharpe and Sortino estimates by this factor.

3.10 Rolling Analytics

jQuantStats computes rolling variants of the Sharpe ratio, Sortino ratio, volatility, and benchmark greeks (alpha and beta) over a configurable look-back window w:

$$\text{SR}_t^{(w)} = \frac{\bar{r}_{t-w+1:t}\,\sqrt{f}}{\hat{\sigma}_{t-w+1:t}}. \tag{35}$$

Rolling computations are implemented via Polars' native `rolling_mean` and `rolling_std` operations, which execute in compiled Rust and avoid Python-level loops.

4 Tilt/Timing Attribution

The Brinson–Hood–Beebower (Brinson et al., 1986) framework decomposes active return into components attributable to asset allocation and security selection. jQuantStats adapts this to a time-series context and refers to the two components as *tilt* and *timing*.

Setup. Let $w_{i,t}$ denote the weight of instrument i on day t as a fraction of AUM, $w^b_{i,t}$ the benchmark weight, and $r_{i,t}$ the return of instrument i. Portfolio return and benchmark return are

$$r^p_t = \sum_i w_{i,t}\, r_{i,t}, \qquad r^b_t = \sum_i w^b_{i,t}\, r_{i,t}. \tag{36}$$

Tilt. The tilt component captures the contribution from holding average overweights or underweights relative to the benchmark:

$$\text{Tilt}_t = \sum_i (\bar{w}_i - \bar{w}^b_i)\, r_{i,t}, \tag{37}$$

where $\bar{w}_i = T^{-1}\sum_t w_{i,t}$ is the time-averaged weight.

Timing. The timing component captures the contribution from dynamic weight deviations around the time-average:

$$\text{Timing}_t = \sum_i (w_{i,t} - \bar{w}_i)\, r_{i,t}. \tag{38}$$

Remark 1. *By construction, $r^p_t - r^b_t = \text{Tilt}_t + \text{Timing}_t + \varepsilon_t$, where ε_t is a residual driven by the benchmark's own active weight deviations. For equal-weight benchmarks with $w^b_{i,t} \equiv 1/N$ this residual vanishes.*

The library exposes this decomposition through `Portfolio.tilt`, `Portfolio.timing`, and `Portfolio.tilt_timing_decomp()`.

5 Transaction Cost Models

jQuantStats implements two independent, additive transaction cost models encapsulated in `CostModel`.

5.1 Model A: Per-Unit Cost

Each time a position changes by $\Delta n_{i,t}$ units, a cost of c_{unit} per unit is incurred:

$$C_t^A = c_{\text{unit}} \sum_i |\Delta n_{i,t}|. \tag{39}$$

This model is appropriate for instruments with fixed brokerage commissions, stamp duty, or exchange fees quoted per share. It is constructed with `CostModel.per_unit(c)`.

5.2 Model B: Turnover Basis Points

Each unit of one-way portfolio turnover incurs a cost of c_{bps} basis points of AUM:

$$C_t^B = \frac{c_{\text{bps}}}{10\,000} \cdot \text{AUM}_t \cdot \frac{\sum_i |\Delta w_{i,t}|}{2}, \tag{40}$$

where $\Delta w_{i,t}$ is the change in weight of instrument i. This model is appropriate for strategies that are sized in terms of portfolio weight and where the dominant cost is the bid–offer spread. It is constructed with `CostModel.turnover_bps(c)`.

Remark 2. *The two models are mutually exclusive at construction time: a `CostModel` may have a non-zero per-unit cost or a non-zero turnover-bps cost, but not both. This constraint avoids accidental double-counting.*

5.3 Cost-Adjusted Returns

`Portfolio.cost_adjusted_returns()` subtracts C_t/AUM_t from the gross return series to produce a net-of-cost return stream on which all downstream metrics are computed. The method `Portfolio.trading_cost_impact_plot(max_bps)` sweeps the turnover-bps parameter from 0 to `max_bps` and plots how key metrics (Sharpe ratio, Calmar ratio, annualised return) degrade with increasing cost, providing practitioners with a breakeven cost analysis.

6 Visualisation and Reporting

6.1 Data Plots

`DataPlots.plot_snapshot()` renders a three-panel dashboard:

1. Cumulative return (linear or log scale, switchable interactively).
2. Underwater drawdown chart.
3. Monthly returns heatmap with calendar layout.

6.2 Portfolio Plots

`PortfolioPlots` exposes the following charts.

Method	Description
`snapshot(log_scale)`	NAV overlay with drawdown subplot
`rolling_sharpe_plot(window)`	Time series of $\mathrm{SR}_t^{(w)}$
`rolling_volatility_plot(window)`	Time series of $\sigma_t^{(w)}$
`annual_sharpe_plot()`	Per-year Sharpe ratio bar chart
`monthly_returns_heatmap()`	Calendar-style return heat map
`correlation_heatmap()`	Cross-instrument correlation matrix
`lead_lag_ir_plot(start, end)`	IR as a function of signal lag ℓ
`lagged_performance_plot(lags)`	Cumulative NAV for each lag ℓ
`smoothed_holdings_performance_plot()`	Smoothed position PnL
`trading_cost_impact_plot(max_bps)`	Metric decay vs. cost sweep

Lead-lag analysis. Given a position matrix $w_{i,t}$ derived from a signal, the lead-lag IR plot re-evaluates portfolio performance under a lag of $\ell \in \{0, 1, \ldots, L\}$ days, replacing $w_{i,t}$ with $w_{i,t-\ell}$. A sharp degradation at small lags confirms that the strategy's edge is genuinely predictive rather than capacity-constrained by execution timing.

6.3 HTML Reports

Both `Portfolio` and `Data` expose a `.report` (or `.reports`) facade. Calling `.to_html(title)` produces a self-contained HTML file—embedding all Plotly charts inline as JavaScript—that can be shared without external dependencies.

7 Annualisation and Frequency Detection

All ratio metrics require a value for f, the number of periods per year. jQuantStats infers f automatically from the median observed gap between consecutive timestamps in the return series according to the following table.

Detected frequency	f
Daily	252
Weekly	52
Bi-weekly	26
Monthly	12
Quarterly	4
Annual	1

The caller may override f by passing `periods=` to any method that accepts it. When computing rolling statistics, the same f is used for annualisation inside each window.

Remark 3. *Using 252 trading days rather than 365 calendar days is standard practice in equity portfolio analytics. Practitioners working with futures or FX returns that accrue daily over weekends may wish to override to 365.*

8 End-to-End Usage Example

The following listing demonstrates a representative workflow: loading data, constructing a `Portfolio`, inspecting key metrics, running attribution, and saving an HTML report.

```python
import polars as pl
from jquantstats import Portfolio, CostModel

# ------------------------------------------------------------------ #
# 1. Load price and position data (example: two instruments)          #
# ------------------------------------------------------------------ #
prices = pl.read_csv("prices.csv", try_parse_dates=True)
# prices has columns: ["Date", "AAPL", "MSFT"]

positions = pl.read_csv("positions.csv", try_parse_dates=True)
# positions has columns: ["Date", "AAPL", "MSFT"]
# values are signed cash notionals in GBP

# ------------------------------------------------------------------ #
# 2. Construct the Portfolio with a 5 bps/side cost model             #
# ------------------------------------------------------------------ #
pf = Portfolio.from_cash_position(
    prices=prices,
    cash_position=positions,
    aum=10_000_000,
    cost_bps=5.0,          # 5 bps of AUM per unit of one-way turnover
)

# ------------------------------------------------------------------ #
# 3. Inspect summary statistics                                       #
# ------------------------------------------------------------------ #
print(pf.stats.summary())

# ------------------------------------------------------------------ #
# 4. Risk-adjusted ratios                                             #
# ------------------------------------------------------------------ #
print("Sharpe     :", round(pf.stats.sharpe(), 3))
print("Sortino    :", round(pf.stats.sortino(), 3))
print("Calmar     :", round(pf.stats.calmar(), 3))
print("Max DD     :", round(pf.stats.max_drawdown(), 4))
print("CVaR 5%    :", round(pf.stats.conditional_value_at_risk(), 4))
print("Kelly      :", round(pf.stats.kelly_criterion(), 4))

# ------------------------------------------------------------------ #
# 5. Benchmark attribution (if a benchmark is available)              #
# ------------------------------------------------------------------ #
bench = pl.read_csv("sp500.csv", try_parse_dates=True)["Return"]
d = pf.data  # bridge to Data API
d_with_bench = d.__class__.from_returns(d.returns, benchmark=bench)
print("Alpha      :", round(d_with_bench.stats.greeks()["alpha"], 4))
print("Beta       :", round(d_with_bench.stats.greeks()["beta"], 4))
print("IR         :", round(d_with_bench.stats.information_ratio(), 3))

# ------------------------------------------------------------------ #
# 6. Tilt / Timing decomposition                                      #
# ------------------------------------------------------------------ #
tilt, timing = pf.tilt, pf.timing
print("Tilt Sharpe  :", round(tilt.stats.sharpe(), 3))
print("Timing Sharpe:", round(timing.stats.sharpe(), 3))
```

```
55
56 # ------------------------------------------------------------------ #
57 # 7. Turnover summary                                                #
58 # ------------------------------------------------------------------ #
59 print(pf.turnover_summary())
60
61 # ------------------------------------------------------------------ #
62 # 8. Interactive plots                                               #
63 # ------------------------------------------------------------------ #
64 pf.plots.snapshot().show()
65 pf.plots.rolling_sharpe_plot(window=63).show()
66 pf.plots.trading_cost_impact_plot(max_bps=20).show()
67
68 # ------------------------------------------------------------------ #
69 # 9. Save self-contained HTML report                                 #
70 # ------------------------------------------------------------------ #
71 pf.report.save("strategy_report.html", title="My Strategy")
```

Listing 3: Complete usage example.

9 Design Decisions and Limitations

9.1 Zero Pandas Dependency

The decision to eliminate pandas from the runtime dependency graph was deliberate. Pandas' object-dtype columns, copy-on-write semantics, and implicit type coercions have historically been a source of silent correctness bugs in financial libraries. Polars enforces strict column types, returns immutable DataFrames by default, and provides a significantly faster execution engine for the columnar operations that dominate portfolio analytics.

9.2 Narwhals Boundary Layer

Although the core is Polars-native, some users maintain existing pipelines in pandas. The Narwhals boundary layer allows both pandas `DataFrame`s and Polars `DataFrame`s to be passed to factory methods. Internally, the library immediately converts to Polars for all computations; the abstraction adds negligible overhead.

9.3 Limitations

- **Single-currency.** The library does not model multi-currency portfolios or FX hedging. All cash flows are assumed to be denominated in a single base currency.
- **Linear cost models only.** Market impact models (e.g. Almgren–Chriss) and non-linear fee schedules are not yet implemented.
- **No live data integration.** jQuantStats is a purely offline analytics library. Real-time data feeds must be managed by the caller.
- **Parametric VaR.** The Cornish–Fisher expansion is an approximation that may understate tail risk for highly non-Gaussian return distributions. Historical or Monte Carlo simulation-based VaR is not implemented.

10 Conclusion

jQuantStats provides a modern, type-safe, and performant foundation for portfolio analytics in Python. By adopting Polars as the compute backend, Plotly as the visualisation layer, and a clean two-class public API, it addresses the limitations of the previous generation of pandas-based tools while retaining full compatibility via the Narwhals boundary layer.

The mathematical definitions in Section 3 give practitioners an unambiguous reference for the precise computation of each metric. The Brinson-style tilt/timing decomposition in Section 4 and the two-factor cost model in Section 5 extend the library beyond simple performance measurement into attribution and cost analysis.

Future development priorities include multi-currency support, non-linear cost modelling, and expanded factor attribution capabilities.

References

Ran Aroussi. Quantstats: Portfolio analytics for quants, 2019. https://github.com/ranaroussi/quantstats.

David H. Bailey and Marcos López de Prado. The sharpe ratio efficient frontier. *Journal of Risk*, 15(2):3–44, 2012.

Gary P. Brinson, L. Randolph Hood, and Gilbert L. Beebower. Determinants of portfolio performance. *Financial Analysts Journal*, 42(4):39–44, 1986.

Edmund A. Cornish and Ronald A. Fisher. Moments and cumulants in the specification of distributions. *Revue de l'Institut International de Statistique*, 5(4):307–320, 1937.

Orris C. Herfindahl. Concentration in the U.S. steel industry. *Dissertation, Columbia University*, 1950.

Michael C. Jensen. The performance of mutual funds in the period 1945–1964. *Journal of Finance*, 23(2):389–416, 1968.

Con Keating and William F. Shadwick. A universal performance measure. *Journal of Performance Measurement*, 6(3):59–84, 2002.

Narwhals Development Team. Narwhals: Lightweight compatibility layer for dataframe libraries, 2024. https://github.com/narwhals-dev/narwhals.

Polars Development Team. Polars: Blazingly fast dataframes in Rust and Python. *GitHub Repository*, 2024. https://github.com/pola-rs/polars.

William F. Sharpe. Mutual fund performance. *Journal of Business*, 39(1):119–138, 1966.

Frank A. Sortino and Lee N. Price. Performance measurement in a downside risk framework. *Journal of Investing*, 3(3):59–64, 1994.

Terry W. Young. Calmar ratio: A smoother tool. *Futures*, 20(1):40, 1991.

A API Quick Reference

A.1 Portfolio — Key Properties and Methods

Property / Method	Description
`.profits`	Per-instrument daily PnL DataFrame
`.profit`	Aggregate daily PnL series
`.nav_accumulated`	Linearly accumulated NAV series
`.nav_compounded`	Compounded NAV series
`.returns`	Daily return series
`.monthly`	Monthly return series
`.drawdown`	Drawdown series
`.highwater`	High-water mark series
`.tilt`	Tilt return stream (Data object)
`.timing`	Timing return stream (Data object)
`.tilt_timing_decomp()`	Full Brinson decomposition table
`.turnover`	Daily one-way turnover series
`.turnover_weekly`	Weekly aggregate turnover
`.turnover_summary()`	Summary statistics of turnover
`.cost_adjusted_returns()`	Net-of-cost return series
`.trading_cost_impact(max_bps)`	Metric table across cost sweep
`.correlation()`	Cross-instrument return correlation matrix
`.stats`	Stats facade (see below)
`.plots`	Plots facade (see Section 6)
`.report`	Report facade

A.2 Stats Facade — Key Methods

Method	Returns
`.summary()`	Comprehensive statistics DataFrame
Return metrics	
`.avg_return()`	Mean period return
`.avg_win()`	Mean positive-period return
`.avg_loss()`	Mean negative-period return
`.best()`	Best single-period return
`.worst()`	Worst single-period return
`.comp()`	Compound cumulative return
`.geometric_mean()`	Geometric mean period return
`.cagr()`	Compound annual growth rate
`.ghpr()`	Geometric holding period return
`.expected_return(aggregate)`	Expected return (optionally aggregated)
`.win_rate()`	Fraction of positive-return periods
`.payoff_ratio()`	Avg win / avg loss
`.profit_factor()`	Gross profit / gross loss
`.gain_to_pain_ratio()`	Cumulative return / sum of losses
`.exposure()`	Fraction of non-zero return periods
Risk and volatility	
`.volatility(series, periods, annualize)`	Realised annualised volatility
`.value_at_risk(series, sigma, alpha)`	Cornish–Fisher VaR
`.conditional_value_at_risk()`	Expected Shortfall

Method	Returns
`.kelly_criterion()`	Optimal Kelly fraction
`.tail_ratio(cutoff)`	Ratio of upper to lower tail
`.skew()`	Return distribution skewness
`.kurtosis()`	Return distribution excess kurtosis
`.outliers(quantile)`	Return outlier periods
`.remove_outliers(quantile)`	Return series with outliers removed
`.outlier_win_ratio(quantile)`	Fraction of returns above upper quantile
`.outlier_loss_ratio(quantile)`	Fraction of returns below lower quantile
Drawdown	
`.max_drawdown(series)`	Maximum drawdown
`.max_drawdown_duration()`	Duration of worst drawdown
`.avg_drawdown(series)`	Mean drawdown over underwater periods
`.drawdown_details()`	Per-drawdown-period table per asset
`.calmar(series, periods)`	Calmar ratio
`.recovery_factor(series)`	Total return / max drawdown
Risk-adjusted ratios	
`.sharpe(series, periods)`	Annualised Sharpe ratio
`.sharpe_variance()`	Asymptotic variance of Sharpe estimate
`.probabilistic_sharpe_ratio(sr*)`	Probabilistic Sharpe ratio
`.probabilistic_sortino_ratio()`	Probabilistic Sortino ratio
`.probabilistic_adjusted_sortino_ratio()`	Probabilistic adjusted Sortino ratio
`.probabilistic_ratio(base)`	Generalised probabilistic ratio
`.sortino(series, periods)`	Annualised Sortino ratio
`.adjusted_sortino()`	Sortino / $\sqrt{2}$
`.smart_sharpe()`	Sharpe with autocorrelation penalty
`.smart_sortino()`	Sortino with autocorrelation penalty
`.omega()`	Omega ratio
`.treynor_ratio()`	Treynor ratio
`.ulcer_index()`	Ulcer Index
`.serenity_index()`	Serenity Index
`.common_sense_ratio()`	Common Sense Ratio
`.cpc_index()`	CPC Index
Benchmark and factor analytics	
`.information_ratio()`	Information ratio vs benchmark
`.greeks()`	Dict with alpha and beta
`.r_squared(series, bench)`	R^2 of return regression
`.up_capture()`	Return capture during up-market periods
`.down_capture()`	Return capture during down-market periods
Autocorrelation and serial dependence	
`.autocorr(lag)`	Pearson autocorrelation at lag ℓ
`.acf(nlags)`	Full autocorrelation function as DataFrame
`.autocorr_penalty()`	Autocorrelation penalty factor
Concentration	
`.hhi_positive()`	HHI concentration of gains
`.hhi_negative()`	HHI concentration of losses
Rolling analytics	
`.rolling_sharpe(window)`	Rolling Sharpe ratio series

Method	Returns
`.rolling_sortino(window)`	Rolling Sortino ratio series
`.rolling_volatility(window)`	Rolling volatility series
`.rolling_greeks(window)`	Rolling alpha/beta vs benchmark
Period breakdown	
`.monthly_returns(eoy, compounded)`	Per-month return table
`.monthly_win_rate()`	Win rate aggregated by month
`.annual_breakdown()`	Per-year statistics table
`.distribution(compounded)`	Return distribution by period type
`.compare(aggregate)`	Strategy vs benchmark return table

A.3 CostModel — Factory Methods

Factory	Description
`CostModel.per_unit(c)`	£c per share traded (Model A)
`CostModel.turnover_bps(c)`	c bps of AUM per unit of one-way turnover (Model B)